45｜阶段实操（8）：构建一个简单的KV server- 配置/测试/监控/CI/CD

2021-12-13 陈天

《陈天 · Rust 编程第一课》 课程介绍 >



讲述：陈天

时长 16:57 大小 15.53M

你好，我是陈天。

终于来到了我们这个 KV server 系列的终章。其实原本 KV server 我只计划了 4 讲，但现在 8 讲似乎都还有些意犹未尽。虽然这是一个“简单”的 KV server，它没有复杂的性能优化 —— 我们只用了一句 unsafe；也没有复杂的生命周期处理 —— 只有零星 'static 标注；更没有支持集群的处理。

然而，如果你能够理解到目前为止的代码，甚至能独立写出这样的代码，那么，你已经具备足够的、能在一线大厂开发的实力了，国内我不是特别清楚，但在北美这边，保守一些地说，300k+ USD 的 package 应该可以轻松拿到。

今天我们就给 KV server 项目收个尾，结合之前梳理的实战中 Rust 项目应该考虑的问题，来聊聊和生产环境有关的一些处理，按开发流程，主要讲五个方面：配置、集成测试、性能测试、测量和监控、CI/CD。

配置

首先在 Cargo.toml 里添加 serde 和 toml。我们计划使用 toml 做配置文件，serde 用来处理配置的序列化和反序列化：

复制代码

```
[dependencies]
...
serde = { version = "1", features = ["derive"] } # 序列化/反序列化
...
toml = "0.5" # toml 支持
...
```

然后来创建一个 src/config.rs，构建 KV server 的配置：

复制代码

```
use crate::KvError;
use serde::{Deserialize, Serialize};
use std::fs;

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct ServerConfig {
    pub general: GeneralConfig,
    pub storage: StorageConfig,
    pub tls: ServerTlsConfig,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct ClientConfig {
    pub general: GeneralConfig,
    pub tls: ClientTlsConfig,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct GeneralConfig {
    pub addr: String,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
#[serde(tag = "type", content = "args")]
```

```rust
pub enum StorageConfig {
    MemTable,
    SledDb(String),
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct ServerTlsConfig {
    pub cert: String,
    pub key: String,
    pub ca: Option<String>,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct ClientTlsConfig {
    pub domain: String,
    pub identity: Option<(String, String)>,
    pub ca: Option<String>,
}

impl ServerConfig {
    pub fn load(path: &str) -> Result<Self, KvError> {
        let config = fs::read_to_string(path)?;
        let config: Self = toml::from_str(&config)?;
        Ok(config)
    }
}

impl ClientConfig {
    pub fn load(path: &str) -> Result<Self, KvError> {
        let config = fs::read_to_string(path)?;
        let config: Self = toml::from_str(&config)?;
        Ok(config)
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn server_config_should_be_loaded() {
        let result: Result<ServerConfig, toml::de::Error> =
            toml::from_str(include_str!("../fixtures/server.conf"));
        assert!(result.is_ok());
    }

    #[test]
    fn client_config_should_be_loaded() {
        let result: Result<ClientConfig, toml::de::Error> =
            toml::from_str(include_str!("../fixtures/client.conf"));
        assert!(result.is_ok());
    }
```

```
77 }
```

你可以看到，在 Rust 下，有了 serde 的帮助，处理任何已知格式的配置文件，是多么容易的一件事情。我们**只需要定义数据结构，并为数据结构使用 Serialize/Deserialize 派生宏，就可以处理任何支持 serde 的数据结构**。

我还写了个 examples/gen_config.rs（你可以自行去查阅它的代码），用来生成配置文件，下面是生成的服务端的配置：

复制代码

```
[general]
addr = '127.0.0.1:9527'

[storage]
type = 'SledDb'
args = '/tmp/kv_server'

[tls]
cert = """
-----BEGIN CERTIFICATE-----\r
MIIBdzCCASmgAwIBAgIICpy02U2yuPowBQYDK2VwMDMxCzAJBgNVBAYMAkNOMRIw\r
EAYDVQQKDAlBY21lIEluYy4xEDAOBgNVBAMMB0FjbWUgQ0EwHhcNMjEwOTI2MDEy\r
NTU5WhcNMjYwOTI1MDEyNTU5WjA6MQswCQYDVQQGDAJDTjESMBAGA1UECgwJQWNt\r
ZSBJbmMuMRcwFQYDVQQDDA5BY21lIEtWIHNlcnZlcjAqMAUGAytlcAMhAK2Z2AjF\r
A0uiltNuCvl6EVFl6tpaS/wJYB5IdWT2IISdo1QwUjAcBgNVHREEFTATghFrdnNl\r
cnZlci5hY21lLmluYzATBgNVHSUEDDAKBggrBgEFBQcDATAMBgNVHRMEBTADAQEA\r
MA8GA1UdDwEB/wQFAwMH4AAwBQYDK2VwA0EASGOmOWFPjbGhXNOmYNCa3lInbgRy\r
iTNtB/5kElnbKkhKhRU7yQ8HTHWWkyU5WGWbOOIXEtYp+5ERUJC+mzP9Bw==\r
-----END CERTIFICATE-----\r
"""
key = """
-----BEGIN PRIVATE KEY-----\r
MFMCAQEwBQYDK2VwBCIEIPMyINaewhXwuTPUufFO2mMt/MvQMHrGDGxgdgfy/kUu\r
oSMDIQCtmdgIxQNLopbTbgr5ehFRZeraWkv8CWAeSHVk9iCEnQ==\r
-----END PRIVATE KEY-----\r
"""
```

有了配置文件的支持，就可以在 lib.rs 下写一些辅助函数，让我们创建服务端和客户端更加简单：

复制代码

```
mod config;
mod error;
```

```rust
mod network;
mod pb;
mod service;
mod storage;

pub use config::*;
pub use error::KvError;
pub use network::*;
pub use pb::abi::*;
pub use service::*;
pub use storage::*;

use anyhow::Result;
use tokio::net::{TcpListener, TcpStream};
use tokio_rustls::client;
use tokio_util::compat::FuturesAsyncReadCompatExt;
use tracing::info;

/// 通过配置创建 KV 服务器
pub async fn start_server_with_config(config: &ServerConfig) -> Result<()> {
    let acceptor =
        TlsServerAcceptor::new(&config.tls.cert, &config.tls.key, config.tls.c

    let addr = &config.general.addr;
    match &config.storage {
        StorageConfig::MemTable => start_tls_server(addr, MemTable::new(), acc
        StorageConfig::SledDb(path) => start_tls_server(addr, SledDb::new(path
    };

    Ok(())
}

/// 通过配置创建 KV 客户端
pub async fn start_client_with_config(
    config: &ClientConfig,
) -> Result<YamuxCtrl<client::TlsStream<TcpStream>>> {
    let addr = &config.general.addr;
    let tls = &config.tls;

    let identity = tls.identity.as_ref().map(|(c, k)| (c.as_str(), k.as_str()))
    let connector = TlsClientConnector::new(&tls.domain, identity, tls.ca.as_d
    let stream = TcpStream::connect(addr).await?;
    let stream = connector.connect(stream).await?;

    // 打开一个 stream
    Ok(YamuxCtrl::new_client(stream, None))
}

async fn start_tls_server<Store: Storage>(
    addr: &str,
    store: Store,
    acceptor: TlsServerAcceptor,
```

```
) -> Result<()> {
    let service: Service<Store> = ServiceInner::new(store).into();
    let listener = TcpListener::bind(addr).await?;
    info!("Start listening on {}", addr);
    loop {
        let tls = acceptor.clone();
        let (stream, addr) = listener.accept().await?;
        info!("Client {:?} connected", addr);

        let svc = service.clone();
        tokio::spawn(async move {
            let stream = tls.accept(stream).await.unwrap();
            YamuxCtrl::new_server(stream, None, move |stream| {
                let svc1 = svc.clone();
                async move {
                    let stream = ProstServerStream::new(stream.compat(), svc1.
                    stream.process().await.unwrap();
                    Ok(())
                }
            });
        });
    }
}
```

有了 start_server_with_config 和 start_client_with_config 这两个辅助函数，我们就可以简化 src/server.rs 和 src/client.rs 了。下面是 src/server.rs 的新代码：

复制代码

```
use anyhow::Result;
use kv6::{start_server_with_config, ServerConfig};

#[tokio::main]
async fn main() -> Result<()> {
    tracing_subscriber::fmt::init();
    let config: ServerConfig = toml::from_str(include_str!("../fixtures/server

    start_server_with_config(&config).await?;

    Ok(())
}
```

可以看到，整个代码简洁了很多。在这个重构的过程中，还有一些其它改动，你可以看 GitHub repo 下 46 讲的 diff_config。

集成测试

之前我们写了很多单元测试，但还没有写过一行集成测试。今天就来写一个简单的集成测试，确保客户端和服务器完整的交互工作正常。

之前提到在 Rust 里，集成测试放在 tests 目录下，每个测试编成单独的二进制。所以首先，我们创建和 src 平行的 tests 目录。然后再创建 tests/server.rs，填入以下代码：

复制代码

```rust
use anyhow::Result;
use kv6::{
    start_client_with_config, start_server_with_config, ClientConfig, CommandR
    ProstClientStream, ServerConfig, StorageConfig,
};
use std::time::Duration;
use tokio::time;

#[tokio::test]
async fn yamux_server_client_full_tests() -> Result<()> {
    let addr = "127.0.0.1:10086";

    let mut config: ServerConfig = toml::from_str(include_str!("../fixtures/se
    config.general.addr = addr.into();
    config.storage = StorageConfig::MemTable;

    // 启动服务器
    tokio::spawn(async move {
        start_server_with_config(&config).await.unwrap();
    });

    time::sleep(Duration::from_millis(10)).await;
    let mut config: ClientConfig = toml::from_str(include_str!("../fixtures/cl
    config.general.addr = addr.into();

    let mut ctrl = start_client_with_config(&config).await.unwrap();
    let stream = ctrl.open_stream().await?;
    let mut client = ProstClientStream::new(stream);

    // 生成一个 HSET 命令
    let cmd = CommandRequest::new_hset("table1", "hello", "world".to_string().
    client.execute_unary(&cmd).await?;

    // 生成一个 HGET 命令
    let cmd = CommandRequest::new_hget("table1", "hello");
    let data = client.execute_unary(&cmd).await?;

    assert_eq!(data.status, 200);
    assert_eq!(data.values, &["world".into()]);

    Ok(())
```

```
42 }
```

可以看到，**集成测试的写法和单元测试其实很类似，只不过我们不需要再使用 #[cfg(test)] 来做条件编译**。

如果你的集成测试比较复杂，需要比较多的辅助代码，那么你还可以在 tests 下 cargo new 出一个项目，然后在那个项目里撰写辅助代码和测试代码。如果你对此感兴趣，可以看 tonic 的集成测试。不过注意了，集成测试和你的 crate 用同样的条件编译，所以在集成测试里，无法使用单元测试中构建的辅助代码。

性能测试

在之前不断完善 KV server 的过程中，你一定会好奇：我们的 KV server 性能究竟如何呢？那来写一个关于 Pub/Sub 的性能测试吧。

基本的想法是我们连上 100 个 subscriber 作为背景，然后看 publisher publish 的速度。

因为 BROADCAST_CAPACITY 有限，是 128，当 publisher 速度太快，而导致 server 不能及时往 subscriber 发送时，server 接收 client 数据的速度就会降下来，无法接收新的 client，整体的 publish 的速度也会降下来，所以这个测试能够了解 server 处理 publish 的速度。

为了确认这一点，我们在 start_tls_server() 函数中，在 process() 之前，再加个 100ms 的延时，人为减缓系统的处理速度：

复制代码

```
async move {
    let stream = ProstServerStream::new(stream.compat(), svc1.clone());
    // 延迟 100ms 处理
    time::sleep(Duration::from_millis(100)).await;
    stream.process().await.unwrap();
    Ok(())
}
```

好，现在可以写性能测试了。

在 Rust 下，我们可以用 criterion 库。它可以处理基本的性能测试，并生成漂亮的报告。所以在 Cargo.toml 中加入：

复制代码

```
[dev-dependencies]
...
criterion = { version = "0.3", features = ["async_futures", "async_tokio", "ht
...
rand = "0.8" # 随机数处理
...

[[bench]]
name = "pubsub"
harness = false
```

最后这个 bench section，描述了性能测试的名字，它对应 benches 目录下的同名文件。

我们创建和 src 平级的 benches，然后再创建 benches/pubsub.rs，添入如下代码：

复制代码

```
use anyhow::Result;
use criterion::{criterion_group, criterion_main, Criterion};
use futures::StreamExt;
use kv6::{
    start_client_with_config, start_server_with_config, ClientConfig, CommandR
    StorageConfig, YamuxCtrl,
};
use rand::prelude::SliceRandom;
use std::time::Duration;
use tokio::net::TcpStream;
use tokio::runtime::Builder;
use tokio::time;
use tokio_rustls::client::TlsStream;
use tracing::info;

async fn start_server() -> Result<()> {
    let addr = "127.0.0.1:9999";
    let mut config: ServerConfig = toml::from_str(include_str!("../fixtures/se
    config.general.addr = addr.into();
    config.storage = StorageConfig::MemTable;

    tokio::spawn(async move {
        start_server_with_config(&config).await.unwrap();
    });

```

```
    Ok(())
}

async fn connect() -> Result<YamuxCtrl<TlsStream<TcpStream>>> {
    let addr = "127.0.0.1:9999";
    let mut config: ClientConfig = toml::from_str(include_str!("../fixtures/cl
    config.general.addr = addr.into();

    Ok(start_client_with_config(&config).await?)
}

async fn start_subscribers(topic: &'static str) -> Result<()> {
    let mut ctrl = connect().await?;
    let stream = ctrl.open_stream().await?;
    info!("C(subscriber): stream opened");
    let cmd = CommandRequest::new_subscribe(topic.to_string());
    tokio::spawn(async move {
        let mut stream = stream.execute_streaming(&cmd).await.unwrap();
        while let Some(Ok(data)) = stream.next().await {
            drop(data);
        }
    });

    Ok(())
}

async fn start_publishers(topic: &'static str, values: &'static [&'static str]
    let mut rng = rand::thread_rng();
    let v = values.choose(&mut rng).unwrap();

    let mut ctrl = connect().await.unwrap();
    let mut stream = ctrl.open_stream().await.unwrap();
    info!("C(publisher): stream opened");

    let cmd = CommandRequest::new_publish(topic.to_string(), vec![(*v).into()]
    stream.execute_unary(&cmd).await.unwrap();

    Ok(())
}

fn pubsub(c: &mut Criterion) {
    // tracing_subscriber::fmt::init();
    // 创建 Tokio runtime
    let runtime = Builder::new_multi_thread()
        .worker_threads(4)
        .thread_name("pubsub")
        .enable_all()
        .build()
        .unwrap();
    let values = &["Hello", "Tyr", "Goodbye", "World"];
    let topic = "lobby";

```

```
78      // 运行服务器和 100 个 subscriber，为测试准备
79      runtime.block_on(async {
80          eprint!("preparing server and subscribers");
81          start_server().await.unwrap();
82          time::sleep(Duration::from_millis(50)).await;
83          for _ in 0..100 {
84              start_subscribers(topic).await.unwrap();
85              eprint!(".");
86          }
87          eprintln!("Done!");
88      });
89
90      // 进行 benchmark
91      c.bench_function("publishing", move |b| {
92          b.to_async(&runtime)
93              .iter(|| async { start_publishers(topic, values).await })
94      });
95  }
96
97  criterion_group! {
98      name = benches;
99      config = Criterion::default().sample_size(10);
100     targets = pubsub
101 }
102
```

大部分的代码都很好理解，就是创建服务器和客户端，为测试做准备。说一下这里面核心的 benchmark 代码：

复制代码

```
c.bench_function("publishing", move |b| {
    b.to_async(&runtime)
        .iter(|| async { start_publishers(topic, values).await })
});
```

对于要测试的代码，我们可以封装成一个函数进行测试。**这里因为要做 async 函数的测试，需要使用 runtime。普通的函数不需要调用 to_async**。对于更多有关 criterion 的用法，可以参考它的文档。

运行 cargo bench 后，会见到如下打印（如果你的代码无法通过，可以参考 repo 里的 diff_benchmark，我顺便做了一点小重构）：

复制代码

```
preparing server and subscribers.....................................
publishing              time:   [419.73 ms 426.84 ms 434.20 ms]
                        change: [-1.6712% +1.0499% +3.6586%] (p = 0.48 > 0.05)
                        No change in performance detected.
```

可以看到，单个 publish 的处理速度要 426ms，好慢！我们把之前在 start_tls_server() 里加的延迟去掉，再次测试：

复制代码

```
preparing server and subscribers.....................................
publishing              time:   [318.61 ms 324.48 ms 329.81 ms]
                        change: [-25.854% -23.980% -22.144%] (p = 0.00 < 0.05)
                        Performance has improved.
```

嗯，这下 324ms，正好是减去刚才加的 100ms。可是这个速度依旧不合理，凭直觉我们感觉一下这个速度，是 Python 这样的语言还正常，如果是 Rust 也太慢了吧？

测量和监控

工业界有句名言：如果你无法测量，那你就无法改进（If you can’t measure it, you can’t improve it）。现在知道了 KV server 性能有问题，但并不知道问题出在哪里。我们需要使用合适的测量方式。

目前，**比较好的端对端的性能监控和测量工具是 jaeger**，我们可以在 KV server/client 侧收集监控信息，发送给 jaeger 来查看在服务器和客户端的整个处理流程中，时间都花费到哪里去了。

之前我们在 KV server 里使用的日志工具是 tracing，不过日志只是它的诸多功能之一，它还能做 instrument，然后配合 opentelemetry 库，我们就可以把 instrument 的结果发送给 jaeger 了。

好，在 Cargo.toml 里添加新的依赖：

复制代码

```
[dependencies]
...
opentelemetry-jaeger = "0.15" # opentelemetry jaeger 支持
```

```
...
tracing-appender = "0.1" # 文件日志
tracing-opentelemetry = "0.15" # opentelemetry 支持
tracing-subscriber = { version = "0.2", features = ["json", "chrono"] } # 日志
```

有了这些依赖后，在 benches/pubsub.rs 里，我们可以在初始化 tracing_subscriber 时，使用 jaeger 和 opentelemetry tracer：

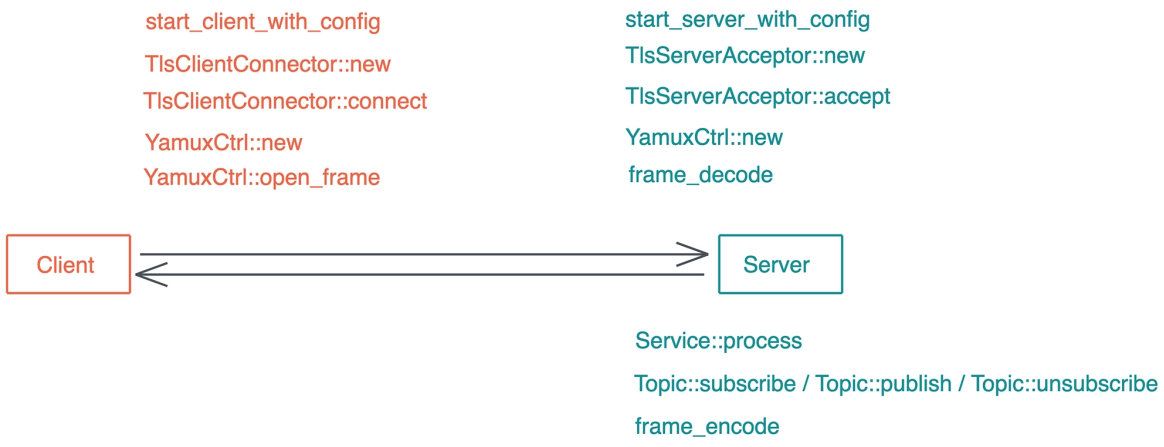
复制代码

```rust
fn pubsub(c: &mut Criterion) {
    let tracer = opentelemetry_jaeger::new_pipeline()
        .with_service_name("kv-bench")
        .install_simple()
        .unwrap();
    let opentelemetry = tracing_opentelemetry::layer().with_tracer(tracer);

    tracing_subscriber::registry()
        .with(EnvFilter::from_default_env())
        .with(opentelemetry)
        .init();

    let root = span!(tracing::Level::INFO, "app_start", work_units = 2);
    let _enter = root.enter();
    // 创建 Tokio runtime
    ...
}
```

设置好 tracing 后，就在系统的主流程上添加相应的 instrument：



极客时间

新添加的代码你可以看 repo 中的 diff_telemetry。注意 instrument 可以用不同的名称，比如，对于 TlsConnector::new() 函数，可以用 `#[instrument(name = "tls_connector_new")]`，这样它的名字辨识度高一些。

为主流程中的函数添加完 instrument 后，你需要先打开一个窗口，运行 jaeger（需要 docker）：

复制代码

```
1 docker run -d -p6831:6831/udp -p6832:6832/udp -p16686:16686 -p14268:14268 jaeg
```

然后带着 RUST_LOG=info 运行 benchmark：

复制代码

```
1 RUST_LOG=info cargo bench
```

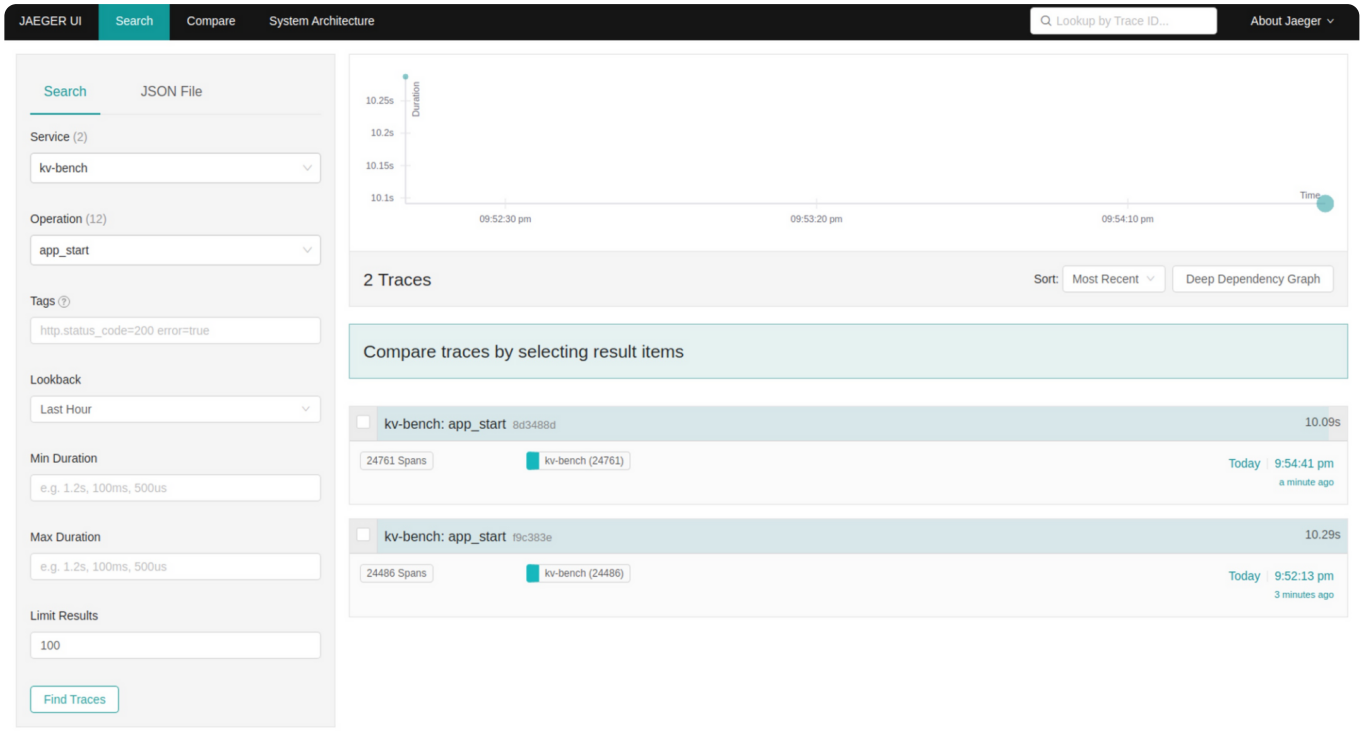
由于我的 OS X 上没装 docker（docker 不支持 Mac，需要 Linux VM 中转），我就在一个 Ubuntu 虚拟机里运行这两条命令：

```
preparing server and subscribers.....................................复制代码
publishing              time:   [1.7464 ms 1.9556 ms 2.2343 ms]
Found 2 outliers among 10 measurements (20.00%)
  1 (10.00%) high mild
  1 (10.00%) high severe
```

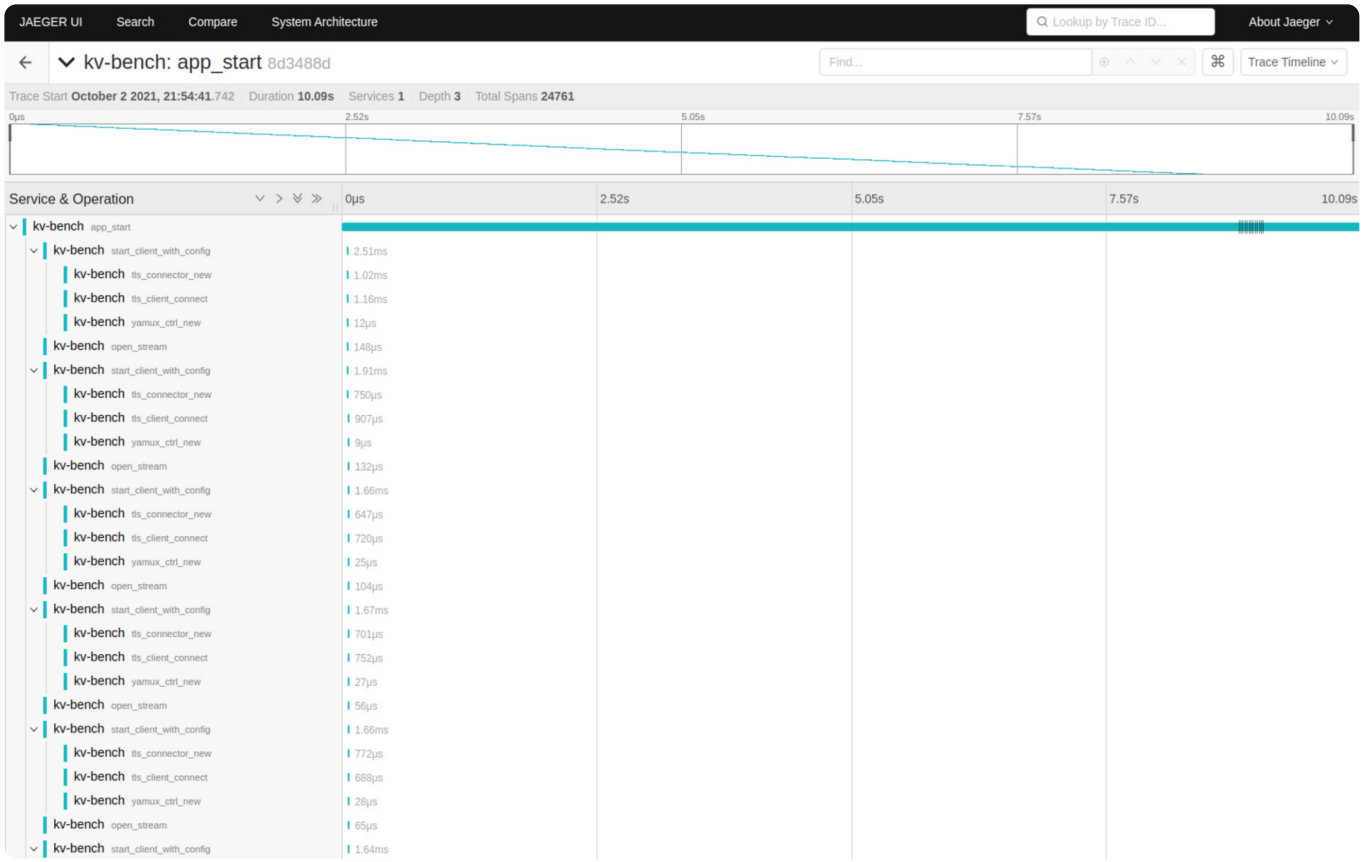
并没有做任何事情，似乎只是换了个系统，性能就提升了很多，这给我们一个 tip：也许问题出在 OS X 和 Linux 系统相关的部分。

不管怎样，已经发送了不少数据给 jaeger，我们到 jaeger 上看看问题出在哪里。

打开 http://localhost:16686/，service 选 kv-bench，Operation 选 app_start，点击“Find Traces”，我们可以看到捕获的 trace。因为运行了两次 benchmark，所以有两个 app_start 的查询结果：



可以看到，每次 start_client_with_config 都要花 1.6-2.5ms，其中有差不多一小半时间花在了 TlsClientConnector::new() 上：



如果说 TlsClientConnector::connect() 花不少时间还情有可原，因为这是整个 TLS 协议的握手过程，涉及到网络调用、包的加解密等。**但 TlsClientConnector::new() 就是加载一些证书、创建 TlsConnector 这个数据结构而已，为何这么慢**？

仔细阅读 TlsClientConnector::new() 的代码，你可以对照注释看：

复制代码

```rust
#[instrument(name = "tls_connector_new", skip_all)]
pub fn new(
    domain: impl Into<String> + std::fmt::Debug,
    identity: Option<(&str, &str)>,
    server_ca: Option<&str>,
) -> Result<Self, KvError> {
    let mut config = ClientConfig::new();

    // 如果有客户端证书，加载之
    if let Some((cert, key)) = identity {
        let certs = load_certs(cert)?;
        let key = load_key(key)?;
        config.set_single_client_cert(certs, key)?;
    }

    // 加载本地信任的根证书链
    config.root_store = match rustls_native_certs::load_native_certs() {
        Ok(store) | Err((Some(store), _)) => store,
```

```
        Err((None, error)) => return Err(error.into()),
    };

    // 如果有签署服务器的 CA 证书，则加载它，这样服务器证书不在根证书链
    // 但是这个 CA 证书能验证它，也可以
    if let Some(cert) = server_ca {
        let mut buf = Cursor::new(cert);
        config.root_store.add_pem_file(&mut buf).unwrap();
    }

    Ok(Self {
        config: Arc::new(config),
        domain: Arc::new(domain.into()),
    })
}
```

可以发现，它的代码唯一可能影响性能的就是加载本地信任的根证书链的部分。这个代码会和操作系统交互，获取信任的根证书链。也许，这就是影响性能的原因之一？

那我们将其简单重构一下。因为根证书链，只有在客户端没有提供用于验证服务器证书的 CA 证书时，才需要，所以可以在没有 CA 证书时，才加载本地的根证书链：

复制代码

```
#[instrument(name = "tls_connector_new", skip_all)]
pub fn new(
    domain: impl Into<String> + std::fmt::Debug,
    identity: Option<(&str, &str)>,
    server_ca: Option<&str>,
) -> Result<Self, KvError> {
    let mut config = ClientConfig::new();

    // 如果有客户端证书，加载之
    if let Some((cert, key)) = identity {
        let certs = load_certs(cert)?;
        let key = load_key(key)?;
        config.set_single_client_cert(certs, key)?;
    }

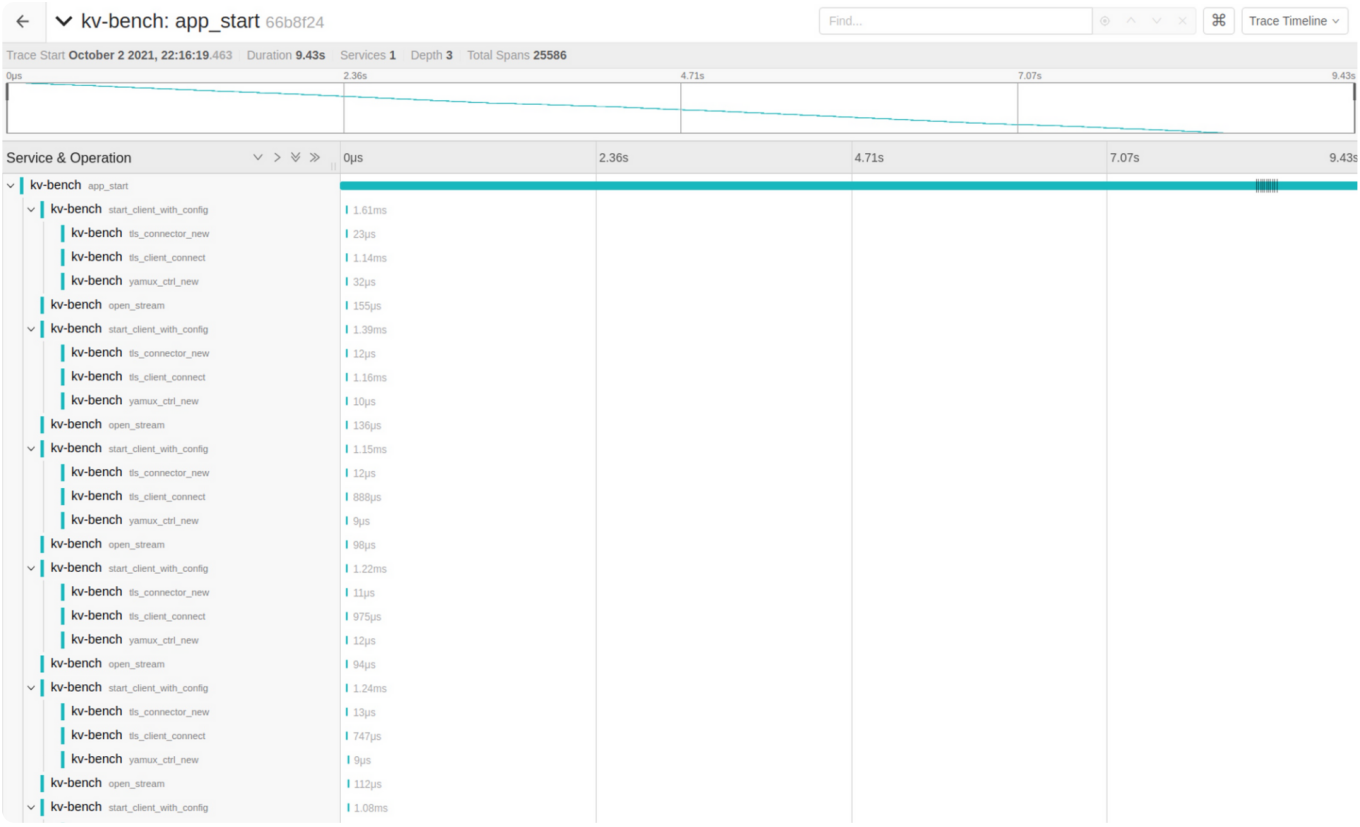
    // 如果有签署服务器的 CA 证书，则加载它，这样服务器证书不在根证书链
    // 但是这个 CA 证书能验证它，也可以
    if let Some(cert) = server_ca {
        let mut buf = Cursor::new(cert);
        config.root_store.add_pem_file(&mut buf).unwrap();
    } else {
        // 加载本地信任的根证书链
        config.root_store = match rustls_native_certs::load_native_certs() {
            Ok(store) | Err((Some(store), _)) => store,
```

```
            Err((None, error)) => return Err(error.into()),
        };
    }

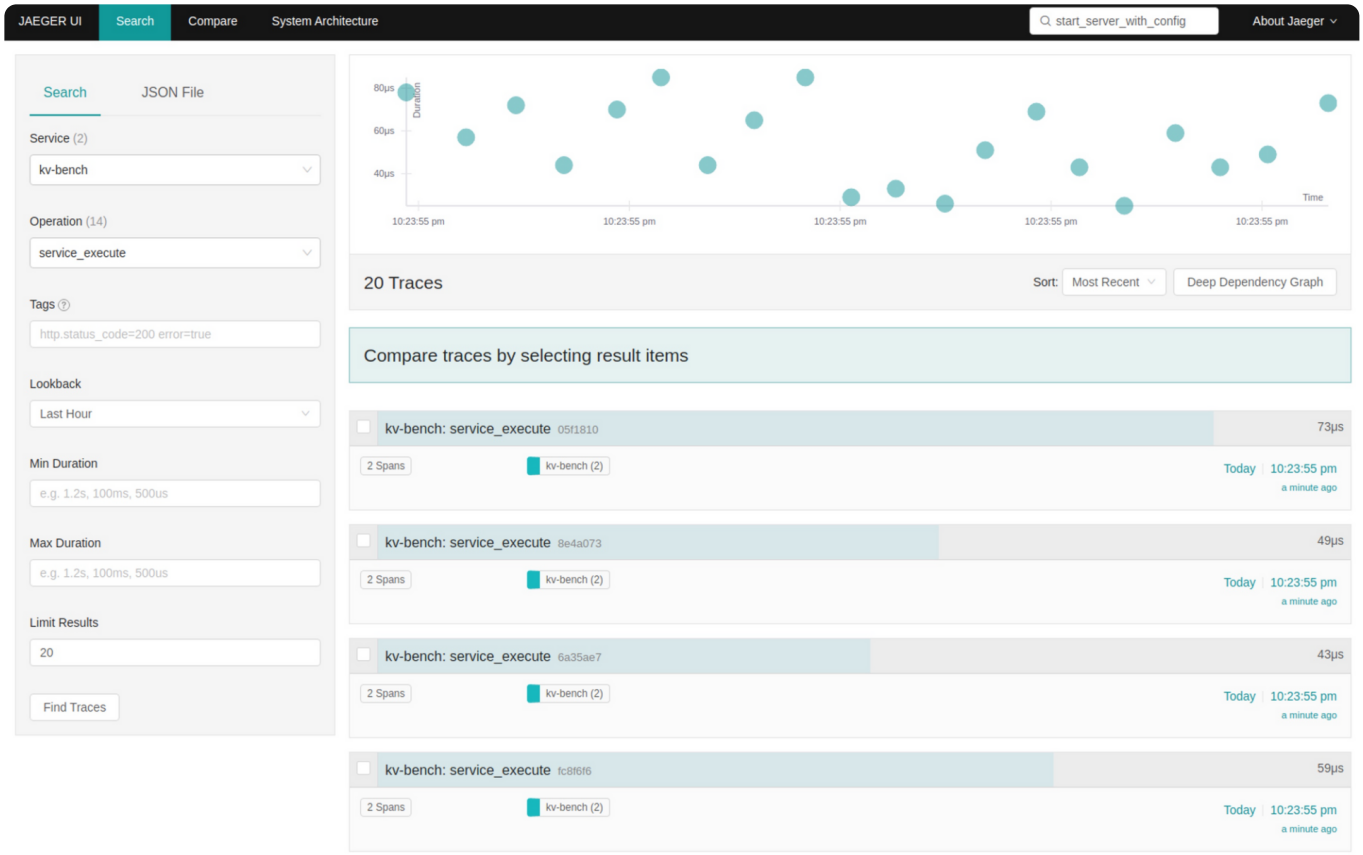
    Ok(Self {
        config: Arc::new(config),
        domain: Arc::new(domain.into()),
    })
}
```

完成这个修改后，我们再运行 `RUST_LOG=info cargo bench`，现在的性能达到了 1.64ms，相比之前的 1.95ms，提升了 16%。

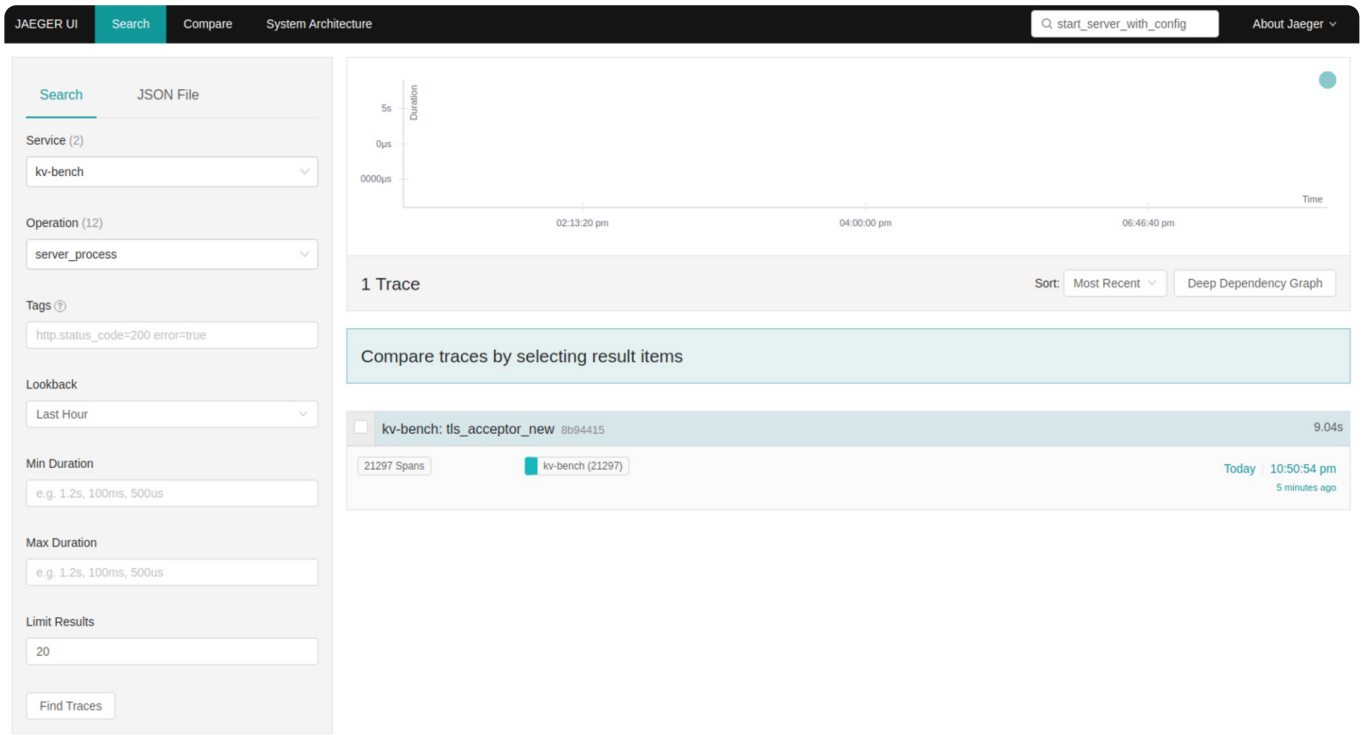
打开 jaeger，看最新的 app_start 结果，发现 TlsClientConnector::new() 所花时间降到了 ~12us 左右。嗯，虽然没有抓到服务器本身的 bug，但客户端的 bug 倒是解决了一个。



至于服务器，如果我们看 Service::execute 的主流程，执行速度在 40-60us，问题不大：



再看服务器的主流程 server_process：



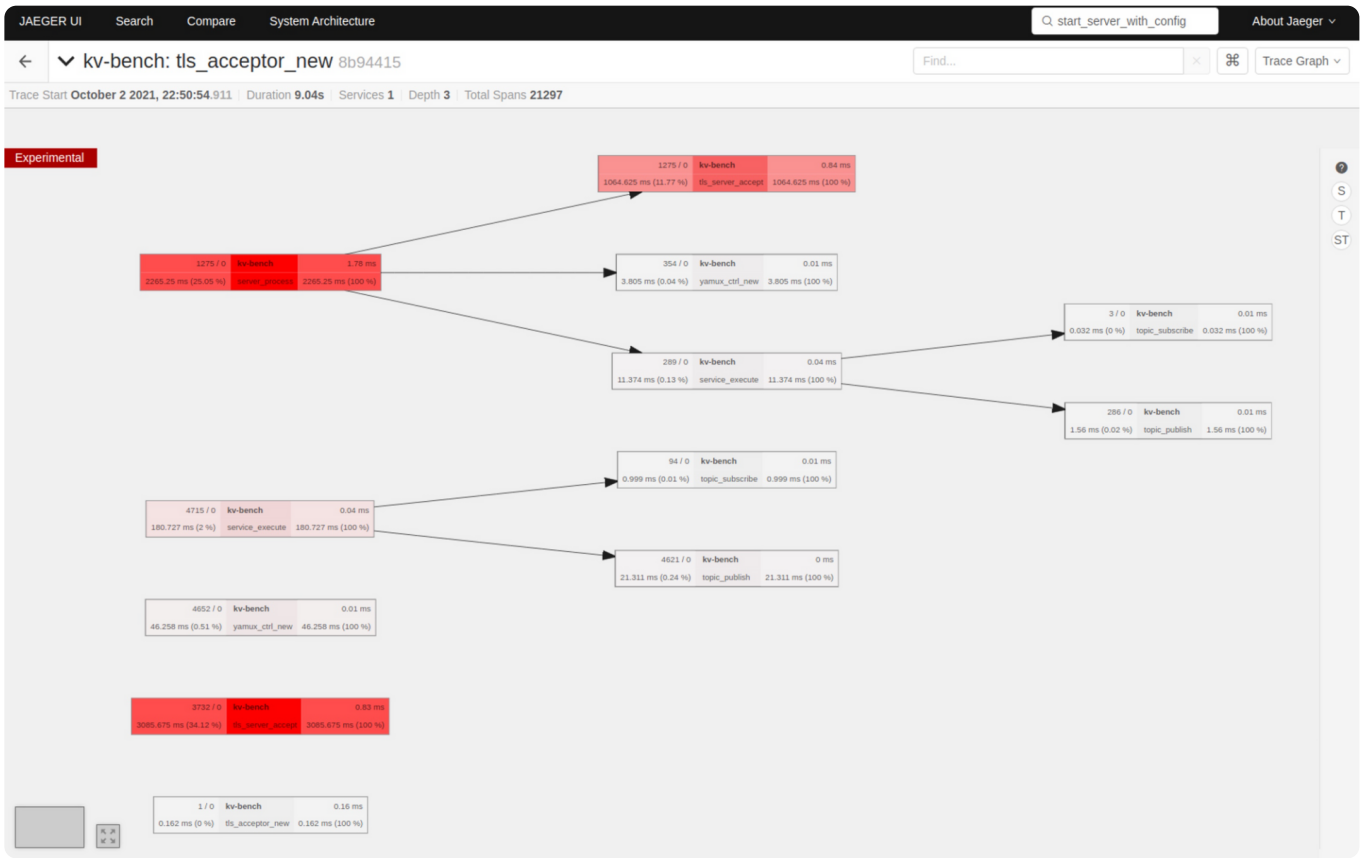
这是我们在 start_tls_server() 里额外添加的 tracing span：

复制代码

```
1 loop {
2
```

```
    let root = span!(tracing::Level::INFO, "server_process");
    let _enter = root.enter();
    ...
```

把右上角的 trace timeline 改成 trace graph，然后点右侧的 time：



可以看到，主要的服务器时间都花在了 TLS accept 上，所以，**目前服务器没有太多值得优化的地方**。

由于 tracing 本身也占用不少 CPU，所以我们直接 `cargo bench` 看看目前的结果：

复制代码

```
preparing server and subscribers.....................................................
publishing              time:   [1.3986 ms 1.4140 ms 1.4474 ms]
                        change: [-26.647% -19.977% -10.798%] (p = 0.00 < 0.05)
                        Performance has improved.
Found 2 outliers among 10 measurements (20.00%)
  2 (20.00%) high severe
```

不加 RUST_LOG=info 后，整体性能到了 1.4ms。这是我在 Ubuntu 虚拟机下的结果。

我们再回到 OS X 下测试，看看 TlsClientConnector::new() 的修改，对 OS X 是否有效：

复制代码

```
preparing server and subscribers.............................................
publishing              time:   [1.4086 ms 1.4229 ms 1.4315 ms]
                        change: [-99.570% -99.563% -99.554%] (p = 0.00 < 0.05)
                        Performance has improved.
```

嗯，在我的 OS X 下，现在整体性能也到了 1.4ms 的水平。这也意味着，在有 100 个 subscribers 的情况下，我们的 KV server 每秒钟可以处理 714k publish 请求；而在 1000 个 subscribers 的情况下，性能在 11.1ms 的水平，也就是每秒可以处理 90k publish 请求：

复制代码

```
publishing              time:   [11.007 ms 11.095 ms 11.253 ms]
                        change: [-96.618% -96.556% -96.486%] (p = 0.00 < 0.05)
                        Performance has improved.
```

你也许会觉得目前 publish 的 value 太小，那换一些更加贴近实际的字符串大小：

复制代码

```
// let values = &["Hello", "Tyr", "Goodbye", "World"];
let base_str = include_str!("../fixtures/server.conf"); // 891 bytes

let values: &'static [&'static str] = Box::leak(
    vec![
        &base_str[..64],
        &base_str[..128],
        &base_str[..256],
        &base_str[..512],
    ]
    .into_boxed_slice(),
);
```

测试结果差不太多：

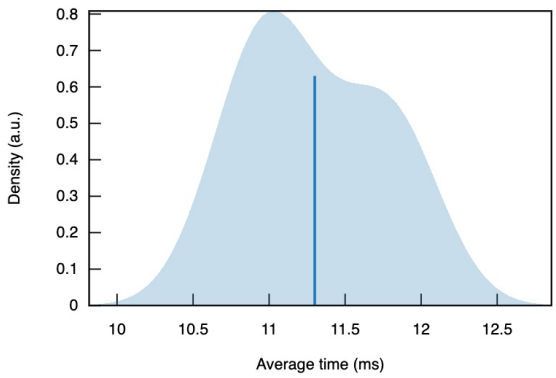
复制代码

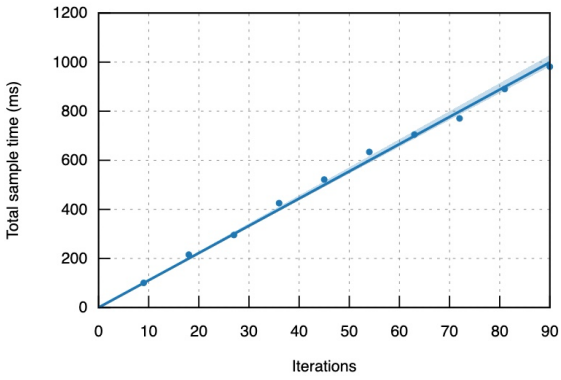
```
publishing              time:   [10.917 ms 11.098 ms 11.428 ms]
                        change: [-0.4822% +2.3311% +4.9631%] (p = 0.12 > 0.05)
```

```
3                    No change in performance detected.
```

criterion 还会生成漂亮的 report，你可以用浏览器打开 ./target/criterion/publishing/report/index.html 查看（名字是 publishing ，因为 benchmark ID 是 publishing）：

publishing





Additional Statistics:

	Lower bound	Estimate	Upper bound
Slope	10.917 ms	11.098 ms	11.428 ms
R²	0.9740541	0.9793810	0.9618567
Mean	11.045 ms	11.300 ms	11.562 ms
Std. Dev.	273.65 us	442.40 us	524.38 us
Median	10.945 ms	11.172 ms	11.739 ms
MAD	66.927 us	507.58 us	695.30 us

Additional Plots:

- Typical
- Mean
- Std. Dev.
- Median
- MAD
- Slope

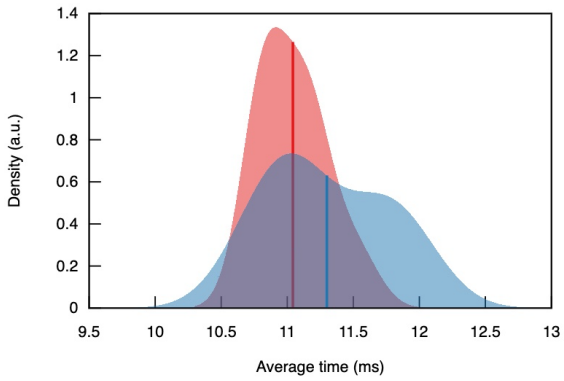
Understanding this report:

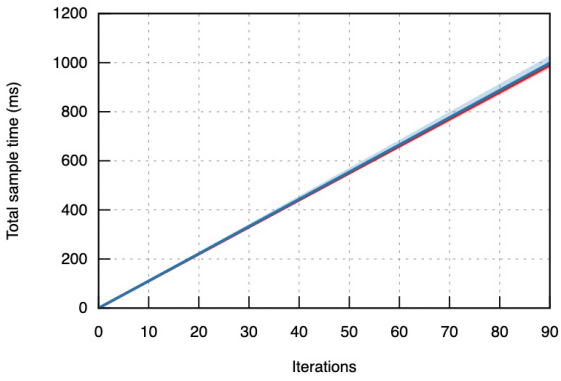
The plot on the left displays the average time per iteration for this benchmark. The shaded region shows the estimated probability of an iteration taking a certain amount of time, while the line shows the mean. Click on the plot for a larger view showing the outliers.

The plot on the right shows the linear regression calculated from the measurements. Each point represents a sample, though here it shows the total time for the sample rather than time per iteration. The line is the line of best fit for these measurements.

See the documentation for more details on the additional statistics.

Change Since Previous Benchmark





Additional Statistics:

	Lower bound	Estimate	Upper bound	
Change in time	-0.4822%	+2.3311%	+4.9631%	(p = 0.12 > 0.05)

No change in performance detected.

Additional Plots:

- Change in mean
- Change in median
- T-Test

好，处理完性能相关的问题，我们来**为 server 添加日志和性能监测的支持**：

复制代码

```rust
use std::env;

use anyhow::Result;
use kv6::{start_server_with_config, RotationConfig, ServerConfig};
use tokio::fs;
use tracing::span;
use tracing_subscriber::{
    fmt::{self, format},
    layer::SubscriberExt,
    prelude::*,
    EnvFilter,
};

#[tokio::main]
async fn main() -> Result<()> {
    // 如果有环境变量，使用环境变量中的 config
    let config = match env::var("KV_SERVER_CONFIG") {
        Ok(path) => fs::read_to_string(&path).await?,
        Err(_) => include_str!("../fixtures/server.conf").to_string(),
    };
    let config: ServerConfig = toml::from_str(&config)?;

    let tracer = opentelemetry_jaeger::new_pipeline()
        .with_service_name("kv-server")
        .install_simple()?;
    let opentelemetry = tracing_opentelemetry::layer().with_tracer(tracer);

    // 添加
    let log = &config.log;
    let file_appender = match log.rotation {
        RotationConfig::Hourly => tracing_appender::rolling::hourly(&log.path,
        RotationConfig::Daily => tracing_appender::rolling::daily(&log.path, "
        RotationConfig::Never => tracing_appender::rolling::never(&log.path, "
    };

    let (non_blocking, _guard1) = tracing_appender::non_blocking(file_appender
    let fmt_layer = fmt::layer()
        .event_format(format().compact())
        .with_writer(non_blocking);

    tracing_subscriber::registry()
        .with(EnvFilter::from_default_env())
        .with(fmt_layer)
        .with(opentelemetry)
        .init();

    let root = span!(tracing::Level::INFO, "app_start", work_units = 2);
```

```
    let _enter = root.enter();

    start_server_with_config(&config).await?;

    Ok(())
}
```

为了让日志能在配置文件中配置，需要更新一下 src/config.rs：

复制代码

```rust
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct ServerConfig {
    pub general: GeneralConfig,
    pub storage: StorageConfig,
    pub tls: ServerTlsConfig,
    pub log: LogConfig,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct LogConfig {
    pub path: String,
    pub rotation: RotationConfig,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum RotationConfig {
    Hourly,
    Daily,
    Never,
}
```

你还需要更新 examples/gen_config.rs。相关的改变可以看 repo 下的 diff_logging。

tracing 和 opentelemetry 还支持 prometheus，你可以使用 opentelemetry-prometheus 来和 prometheus 交互，如果有兴趣，你可以自己深入研究一下。

CI/CD

为了讲述方便，我把 CI/CD 放在最后，但 CI/CD 应该是在一开始的时候就妥善设置的。

先说 CI 吧。这个课程的 repo tyrchen/geektime-rust 在一开始就设置了 github action，每次 commit 都会运行：

代码格式检查：cargo fmt

依赖 license 检查：cargo deny

linting：cargo check 和 cargo clippy

单元测试和集成测试：cargo test

生成文档：cargo doc

github action 配置如下，供你参考：

复制代码

```yaml
name: build

on:
  push:
    branches:
      - master
  pull_request:
    branches:
      - master

jobs:
  build-rust:
    strategy:
      matrix:
        platform: [ubuntu-latest, windows-latest]
    runs-on: ${{ matrix.platform }}
    steps:
      - uses: actions/checkout@v2
      - name: Cache cargo registry
        uses: actions/cache@v1
        with:
          path: ~/.cargo/registry
          key: ${{ runner.os }}-cargo-registry
      - name: Cache cargo index
        uses: actions/cache@v1
        with:
          path: ~/.cargo/git
          key: ${{ runner.os }}-cargo-index
      - name: Cache cargo build
        uses: actions/cache@v1
        with:
          path: target
          key: ${{ runner.os }}-cargo-build-target
      - name: Install stable
        uses: actions-rs/toolchain@v1
        with:
```

```
          profile: minimal
          toolchain: stable
          override: true
      - name: Check code format
        run: cargo fmt -- --check
      - name: Check the package for errors
        run: cargo check --all
      - name: Lint rust sources
        run: cargo clippy --all-targets --all-features --tests --benches -- -D
      - name: Run tests
        run: cargo test --all-features -- --test-threads=1 --nocapture
      - name: Generate docs
        run: cargo doc --all-features --no-deps
      - name: Deploy docs to gh-page
        uses: peaceiris/actions-gh-pages@v3
        with:
          github_token: ${{ secrets.GITHUB_TOKEN }}
          publish_dir: ./target/doc
```

除此之外，我们还可以在每次 push tag 时做 release：

复制代码

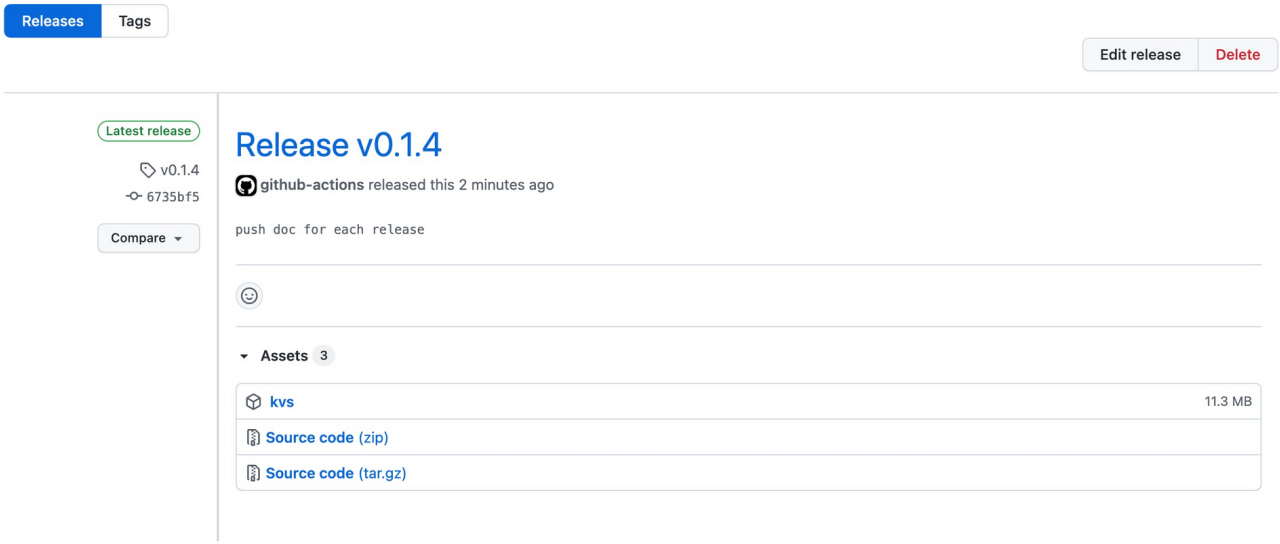
```
name: release

on:
  push:
    tags:
      - "v*" # Push events to matching v*, i.e. v1.0, v20.15.10

jobs:
  build:
    name: Upload Release Asset
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest]
    steps:
      - name: Cache cargo registry
        uses: actions/cache@v1
        with:
          path: ~/.cargo/registry
          key: ${{ runner.os }}-cargo-registry
      - name: Cache cargo index
        uses: actions/cache@v1
        with:
          path: ~/.cargo/git
          key: ${{ runner.os }}-cargo-index
      - name: Cache cargo build
        uses: actions/cache@v1
```

```
      with:
        path: target
        key: ${{ runner.os }}-cargo-build-target
    - name: Checkout code
      uses: actions/checkout@v2
      with:
        token: ${{ secrets.GH_TOKEN }}
        submodules: recursive
    - name: Build project
      run: |
        make build-release
    - name: Create Release
      id: create_release
      uses: actions/create-release@v1
      env:
        GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      with:
        tag_name: ${{ github.ref }}
        release_name: Release ${{ github.ref }}
        draft: false
        prerelease: false
    - name: Upload asset
      id: upload-kv-asset
      uses: actions/upload-release-asset@v1
      env:
        GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      with:
        upload_url: ${{ steps.create_release.outputs.upload_url }}
        asset_path: ./target/release/kvs
        asset_name: kvs
        asset_content_type: application/octet-stream
    - name: Set env
      run: echo "RELEASE_VERSION=${GITHUB_REF#refs/*/}" >> $GITHUB_ENV
    - name: Deploy docs to gh-page
      uses: peaceiris/actions-gh-pages@v3
      with:
        github_token: ${{ secrets.GITHUB_TOKEN }}
        publish_dir: ./target/doc/simple_kv
        destination_dir: ${{ env.RELEASE_VERSION }}
```

这样，每次 push tag 时，都可以打包出来 Linux 的 kvs 版本：



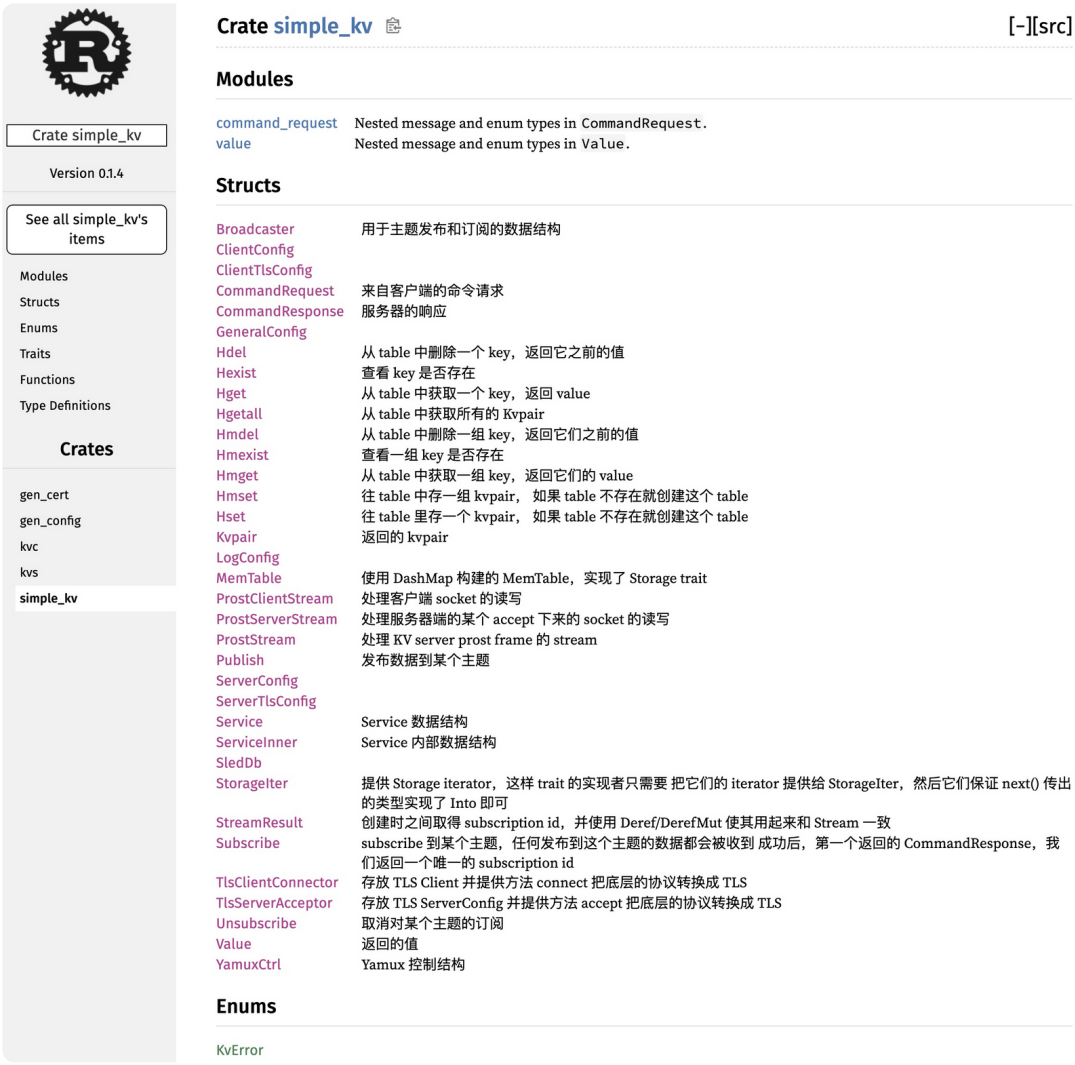
如果你不希望直接使用编译出来的二进制，也可以打包成 docker，在 Kubernetes 下使用。

在做 CI 的过程中，我们也可以触发 CD，比如：

> PR merge 到 master，在 build 完成后，触发 dev 服务器的部署，团队内部可以尝试；
>
> 如果 release tag 包含 alpha，在 build 完成后，触发 staging 服务器的部署，公司内部可以使用；
>
> 如果 release tag 包含 beta，在 build 完成后，触发 beta 服务器的部署，beta 用户可以使用；
>
> 正式的 release tag 会触发生产环境的滚动升级，升级覆盖到的用户可以使用。

一般来说，每家企业都有自己的 CI/CD 的工具链，这里为了展示方便，我们演示了如何使用 github action 对 Rust 代码做 CI，你可以按照自己的需要来处理。

在刚才的 action 代码中，还编译并上传了文档，所以我们可以通过 github pages 很方便地访问文档：



Crate simple_kv [-][src]

Modules

command_request	Nested message and enum types in `CommandRequest`.
value	Nested message and enum types in `Value`.

Structs

Broadcaster	用于主题发布和订阅的数据结构
ClientConfig	
ClientTlsConfig	
CommandRequest	来自客户端的命令请求
CommandResponse	服务器的响应
GeneralConfig	
Hdel	从 table 中删除一个 key，返回它之前的值
Hexist	查看 key 是否存在
Hget	从 table 中获取一个 key，返回 value
Hgetall	从 table 中获取所有的 Kvpair
Hmdel	从 table 中删除一组 key，返回它们之前的值
Hmexist	查看一组 key 是否存在
Hmget	从 table 中获取一组 key，返回它们的 value
Hmset	往 table 中存一组 kvpair，如果 table 不存在就创建这个 table
Hset	往 table 里存一个 kvpair，如果 table 不存在就创建这个 table
Kvpair	返回的 kvpair
LogConfig	
MemTable	使用 DashMap 构建的 MemTable，实现了 Storage trait
ProstClientStream	处理客户端 socket 的读写
ProstServerStream	处理服务器端的某个 accept 下来的 socket 的读写
ProstStream	处理 KV server prost frame 的 stream
Publish	发布数据到某个主题
ServerConfig	
ServerTlsConfig	
Service	Service 数据结构
ServiceInner	Service 内部数据结构
SledDb	
StorageIter	提供 Storage iterator，这样 trait 的实现者只需要 把它们的 iterator 提供给 StorageIter，然后它们保证 next() 传出的类型实现了 Into 即可
StreamResult	创建时之间取得 subscription id，并使用 Deref/DerefMut 使其用起来和 Stream 一致
Subscribe	subscribe 到某个主题，任何发布到这个主题的数据都会被收到 成功后，第一个返回的 CommandResponse，我们返回一个唯一的 subscription id
TlsClientConnector	存放 TLS Client 并提供方法 connect 把底层的协议转换成 TLS
TlsServerAcceptor	存放 TLS ServerConfig 并提供方法 accept 把底层的协议转换成 TLS
Unsubscribe	取消对某个主题的订阅
Value	返回的值
YamuxCtrl	Yamux 控制结构

Enums

KvError

Crate simple_kv
Version 0.1.4
See all simple_kv's items
Modules
Structs
Enums
Traits
Functions
Type Definitions
Crates
gen_cert
gen_config
kvc
kvs
simple_kv

小结

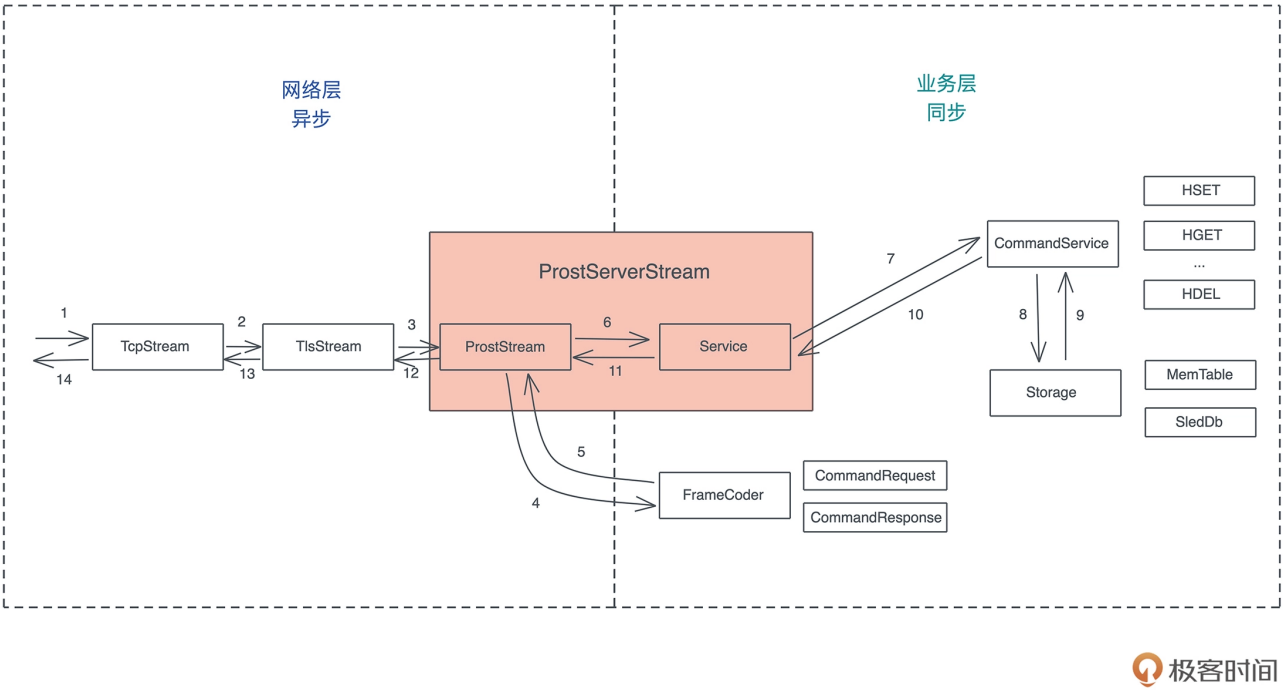
我们的 KV server 之旅就到此为止了。在整整 7 堂课里，我们一点点从零构造了一个完整的 KV server，包括注释在内，撰写了近三千行代码：

复制代码

```
❯ tokei .
-------------------------------------------------------------------------------
 Language            Files        Lines         Code     Comments       Blanks
-------------------------------------------------------------------------------
 Makefile                1           24           16            1            7
 Markdown                1            7            7            0            0
 Protocol Buffers        1          119           79           23           17
 Rust                   25         3366         2730          145          491
 TOML                    2          268          107          142           19
-------------------------------------------------------------------------------
 Total                  30         3784         2939          311          534
-------------------------------------------------------------------------------
```

这是一个非常了不起的成就！我们应该为自己感到自豪！

在这个系列里，我们大量使用 trait 和泛型，构建了很多复杂的数据结构；还为自己的类型实现了 AsyncRead / AsyncWrite / Stream / Sink 这些比较高阶的 trait。通过良好的设计，我们把网络层和业务层划分地非常清晰，网络层的变化不会影响到业务层，反之亦然：



极客时间

我们还模拟了比较真实的开发场景，通过大的需求变更，引发了一次不小的代码重构。

最终，通过性能测试，发现了一个客户端实现的小 bug。在处理这个 bug 的时候，我们欣喜地看到，Rust 有着非常强大的测试工具链，除了我们使用的单元测试、集成测试、性能测试，Rust 还支持模糊测试（fuzzy testing）和基于特性的测试（property testing）。

对于测试过程中发现的问题，Rust 有着非常完善的 tracing 工具链，可以和整个 opentelemetry 生态系统（包括 jaeger、prometheus 等工具）打通。我们就是通过使用 jaeger 找到并解决了问题。除此之外，Rust tracing 工具链还支持生成 flamegraph，篇幅关系，没有演示，你感兴趣的话可以试试。

最后，我们完善了 KV server 的配置、日志以及 CI。完整的代码我放在了 github.com/tyrchen/simple-kv 上，欢迎查看最终的版本。

希望通过这个系列，你对如何使用 Rust 的特性来构造应用程序有了深度的认识。我相信，如果你能够跟得上这个系列的节奏，另外如果遇到新的库，用第 20 讲阅读代码的方式快速掌握，那么，大部分 Rust 开发中的挑战，对你而言都不是难事。

思考题

我们目前并未对日志做任何配置。一般来说，怎么做日志，会有相应的开关以及日志级别，如果希望能通过如下的配置记录日志，该怎么做？试试看：

复制代码

```
[log]
enable_log_file = true
enable_jaeger = false
log_level = 'info'
path = '/tmp/kv-log'
rotation = 'Daily'
```

欢迎在留言区分享自己做 KV server 系列的想法和感悟。你已经完成了第 46 次打卡，我们下节课见。

分享给需要的人，Ta订阅后你可得 20 元现金奖励

生成海报并分享

赞 5 提建议

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 44 | 数据处理：应用程序和数据如何打交道？

下一篇 46 | 软件架构：如何用 Rust 架构复杂系统？



精选留言 (6)

写留言

Rayjun
2021-12-13

老师的工程底子是真的厚，我感觉整个课程下来，学 rust 是其次，更重要的是学到了老师做工程的方法，太棒了

展开

共 1 条评论 > 4

乌龙猹
2021-12-13

这确实是 Rust 编程的第一课 但确是每一个务实程序员的必修课 在做一项工程时，老师的思维方式拆解问题的方法都毫无保留的分享出来 值得我们反复去阅读 反复理解 等什么时候能达到融会贯通 举一反三的时候 那时候就离 300k 的 package 不远了

展开

3

吴云阁
2021-12-15

这节课的内容不仅仅是rust编程，更是毫无保留的分享工程方法。老师不仅仅是一个极其

优秀的工程师，也是一个极其优秀的教学者。非常值得学习。

展开



罗杰

2021-12-13

实操项目 120 分，我要抽出连续时间好好完成这个项目，该专栏真的值得看十遍。



楠楠嘻嘻

2021-12-13

确实厉害！真的是rust in action！

展开



yyxxccc

2021-12-13

看到300k的pkg，又增加了持续反复啃的动力

展开